

AstraCode Qiskit Fall Fest 2026 Hackathon: Quantum Quest

Organised by the AstraCode community as part of Qiskit Fall Fest 2026

Welcome

Qiskit Fall Fest is a global series of community-run events where students, developers, and curious beginners learn quantum computing together using Qiskit, IBM's open-source quantum toolkit. This hackathon is AstraCode's part of that effort.

You do not need to be a quantum expert to take part. In one event, you will go from "what is a qubit?" to running your very own quantum program.

The Challenge

Build a working prototype that uses Qiskit to demonstrate one quantum-computing concept.

Your project should include:

- A runnable notebook or application
- A short explanation of the quantum idea behind it
- A visualization

Keep your team focused on **one concept** and explain it well. A simple project that is clearly understood beats a complicated one nobody can follow. Simulators are fully allowed.

Choose Your Project

Pick one of the four projects below.

1. Quantum Coin Flipper

Build a quantum coin-flipping program using a Hadamard gate. Explain how measurement creates a random result.

Concepts: qubits, superposition, the Hadamard gate, measurement.

Difficulty: Easiest. A good first project for teams with no quantum experience.

What to build

- A circuit with one qubit and one classical bit: apply a Hadamard gate, then measure.
- Run it once for a single flip, then many times (for example 1,000 shots) to see the pattern.
- Show the results as counts and a bar chart. They should land close to 50% heads and 50% tails.

Suggested steps

1. Create a one-qubit circuit and add the Hadamard gate.
2. Add a measurement.
3. Run it on a simulator for 1 shot, then 1,000 shots.
4. Plot a histogram and label 0 as heads and 1 as tails.

Questions your explanation should answer

- What does the Hadamard gate do to the qubit before it is measured?
- Why does measurement give a single 0 or 1 rather than both?
- Why is the result not exactly 50/50 every time?

Stretch ideas

- Make a biased coin using a rotation gate.
- Flip several coins at once.
- Wrap it in a small heads-or-tails game.
- Show the qubit's state on a Bloch sphere before and after the gate.

2. Quantum Dice Roller

Create a quantum circuit that generates fair dice rolls from measured qubits. Include a visualization of the results.

Concepts: multiple qubits, probability, measurement outcomes, histograms.

Difficulty: Easy. Builds on the coin flip by using several qubits together.

What to build

- A circuit that puts three qubits into superposition and measures them, giving a number from 0 to 7.
- A rule that turns that number into a six-sided die roll: outcomes 0 to 5 become rolls 1 to 6, and 6 or 7 means "roll again".
- A chart of at least 600 rolls, compared with the expected 1 in 6 chance (about 16.7%) for each face.

Suggested steps

1. Create a three-qubit circuit and apply a Hadamard gate to each qubit.
2. Measure all three qubits and read the result as a binary number.
3. Add the re-roll rule for 6 and 7.
4. Repeat many times and plot how often each face appears.

Questions your explanation should answer

- Why do three qubits give 8 outcomes, and why does that not fit a six-sided die directly?
- Why does re-rolling keep the die fair?
- How close to equal should the counts be, and why are they never perfect?

Stretch ideas

- Roll two dice and plot the totals.
- Build a 20-sided die using five qubits.
- Make a simple dice game, such as higher-or-lower.
- Animate the rolls as they come in.

3. Bell State Explorer

Build an interactive notebook demonstrating entanglement with Bell states. Show how measuring one qubit affects the other.

Concepts: entanglement, Bell states, correlated measurements.

Difficulty: Medium. Introduces two-qubit gates and the idea that makes quantum computing different.

What to build

- A two-qubit circuit that creates a Bell state: a Hadamard gate on the first qubit, then a CNOT gate from the first qubit to the second.
- An interactive notebook where the user can choose the number of shots, or pick which of the four Bell states to create, and see the results update.
- A clear demonstration that the two measurements are linked: for the standard Bell state, you only ever see 00 or 11, never 01 or 10.

Suggested steps

1. Build the Bell circuit and draw it.
2. Measure both qubits and run it with many shots.
3. Plot the counts and point out which outcomes never appear.
4. Add simple controls, such as notebook widgets, so a user can change settings and re-run.

Questions your explanation should answer

- Why do the results only ever show matching pairs?
- How is this different from two coins that were simply set to match in advance?
- Does measuring one qubit let you send a message to the other? (Hint: look at what each qubit gives on its own.)

Stretch ideas

- Build all four Bell states and compare their results.
- Measure in a different basis by adding a Hadamard gate before measuring.
- Add noise to the simulation and watch the “impossible” outcomes start to appear.
- Show the circuit diagram and the results side by side in one visual.

4. Quantum Random Number Generator

Use Qiskit to generate random numbers and compare them with Python’s classical random-number generator.

Concepts: quantum randomness, comparing results, simple statistics.

Difficulty: Easy to medium. The coding is simple, and the comparison is where the thinking happens.

What to build

- A program that uses qubits in superposition to produce random bits, then combines them into random numbers (for example, 8 qubits give a number from 0 to 255).
- The same amount of random data generated with Python’s built-in random module.
- A side-by-side comparison, such as counts of 0s and 1s, a histogram of the numbers, and the longest run of identical bits.

Suggested steps

1. Build a circuit with a Hadamard gate on each qubit and measure.
2. Collect at least 1,000 random bits and turn them into numbers.
3. Generate the same amount with Python’s random module.
4. Compare both with simple charts and statistics.

Questions your explanation should answer

- Can you tell the quantum results apart from the Python results just by looking?
- What tests could you run to check that the numbers look random?
- A simulator runs on a normal computer, so how “quantum” is its randomness? Real quantum randomness comes from running on actual quantum hardware.

Stretch ideas

- Draw each set of bits as a black-and-white image and compare the two pictures.
- Run on real IBM Quantum hardware and compare it with the simulator.
- Build a small tool, such as a random password or lottery number generator.
- Run a basic frequency test and report the result honestly, even if there is no visible difference.

What to Submit

1. **A runnable Qiskit notebook or application**, with clear comments in the code
2. **A short explanation** of the quantum idea in plain language (half a page is enough)
3. **A visualization or example**, such as a circuit diagram or results chart, that a beginner can understand

Rules

- Teams of 5 people. No prior quantum experience is required.
- Use Qiskit and Python. Simulators are encouraged, and real IBM Quantum hardware is a bonus.
- Starter notebooks and Qiskit tutorials may be used as a base, but your explanation and the way you present the idea must be your own.
- Every team member should be able to explain what the circuit does.
- Stay focused on one concept rather than trying to cover everything.

Judging Criteria (100 points)

Projects are judged mainly on clarity, learning value, creativity, and a working demo, not on advanced quantum knowledge.

Criteria	What judges look for	Points
Clarity	Could a complete beginner follow the explanation?	30
Learning value	Does the project teach the quantum concept well?	25
Working demo	Does the notebook run and show real results?	25
Creativity	Is the idea, visual, or presentation fresh and engaging?	20

Bonus Points

- **Hardware Hero (+5):** Ran your circuit on a real IBM quantum computer
- **Noise Detective (+5):** Compared simulator results with real or noisy-device results
- **Best Explainer (+5):** Made the concept easy for anyone to understand
- **Clean Code (+5):** Clear, readable, well-documented notebook

Helpful Resources

- IBM Quantum Learning and the Qiskit documentation
- Qiskit Fall Fest workshop notebooks
- AstraCode mentors, available throughout the event

“Start with one qubit. Build something quantum.”